

Support d'oral - Projet MetaVerse Unity

1. Présentation rapide du projet

Le projet est une scène Unity de type mini monde ouvert multijoueur. L'objectif était d'améliorer l'expérience de jeu autour de plusieurs axes :

- déplacement des personnages plus naturel ;
- caméra contrôlable à la souris ;
- interactions entre joueurs avec combat et effets de dégâts ;
- objets bonus à ramasser avec score et bonus de vitesse ;
- véhicules utilisables par les joueurs ;
- circulation autonome avec feux rouges, obstacles et klaxon ;
- ambiance sonore de ville ;
- skins de personnages aléatoires sans doublon ;
- états réseau préparés pour synchroniser les joueurs et les bonus.

L'idée générale est de transformer une scène de démonstration en une expérience plus proche d'un petit jeu urbain, avec des règles de gameplay visibles et compréhensibles.

2. Déplacement et caméra

Au départ, le déplacement était moins intuitif : le personnage pouvait tourner sans réellement se déplacer dans la direction attendue. Le déplacement a été modifié pour suivre l'orientation de la caméra.

Quand le joueur appuie sur avancer, le personnage avance maintenant dans la direction regardée par la caméra, pas seulement dans l'axe global de la scène.

Extrait important :

```
Vector3 GetMovementDirection(Vector2 input)
{
    if (!UseCameraRelativeMovement || Camera.main == null) {
        return new Vector3(input.x * StrafeSpeed, 0f, input.y * WalkSpeed);
    }

    Transform cameraTransform = Camera.main.transform;
    Vector3 cameraForward = Vector3.ProjectOnPlane(cameraTransform.forward,
    Vector3.up).normalized;
    Vector3 cameraRight = Vector3.ProjectOnPlane(cameraTransform.right, Vector3.up).normalized;

    return cameraRight * input.x * StrafeSpeed + cameraForward * input.y * WalkSpeed;
}
```

Ce code utilise `ProjectOnPlane` pour ignorer l'inclinaison verticale de la caméra. On garde seulement la direction horizontale. Cela évite qu'un personnage avance vers le sol ou vers le ciel si la caméra est inclinée.

Le personnage regarde aussi dans la direction où il se déplace :

```
if (movement.sqrMagnitude > 0.001f) {
    Quaternion targetRotation = Quaternion.LookRotation(movement.normalized, Vector3.up);
    Quaternion nextRotation = Quaternion.RotateTowards(rb.rotation, targetRotation, TurnSpeed *
    Time.fixedDeltaTime);
    rb.MoveRotation(nextRotation);
}
```

```
}
```

Point à dire à l'oral : on utilise `Rigidbody.MoveRotation`` et `Rigidbody.MovePosition`` dans `FixedUpdate``, ce qui est plus adapté à un objet contrôlé par la physique Unity.

3. Combat entre joueurs

Un système de combat simple a été ajouté. Chaque joueur peut attaquer avec une touche. Si un autre personnage est dans la zone d'attaque, il reçoit un effet de dégâts.

Effets ajoutés :

- ralentissement temporaire après un coup ;
- petite poussée avec `AddForce`` ;
- couleur rouge pour montrer les dégâts ;
- animation de hit ;
- combo : après plusieurs coups, le joueur tombe au sol puis se relève.

Extrait :

```
void ReceiveHit(Vector3 attackerPosition, float duration)
{
    if (isKnockedDown) { return; }

    slowedUntil = Mathf.Max(slowedUntil, Time.time + duration);

    Vector3 knockbackDirection = transform.position - attackerPosition;
    knockbackDirection.y = 0f;
    if (knockbackDirection.sqrMagnitude > 0.001f) {
        rb.AddForce(knockbackDirection.normalized * AttackKnockback, ForceMode.VelocityChange);
    }
}
```

Point à dire à l'oral : on ne crée pas un système de vie complet, mais on crée déjà une boucle de gameplay lisible : attaquer, gêner l'autre joueur, lui faire perdre du temps, puis profiter de ce temps pour récupérer des cubes.

Le combo est contrôlé par :

```
public int HitsBeforeKnockDown = 3;
public float HitComboWindow = 2f;
```

Cela permet de dire : "si un joueur reçoit plusieurs coups dans une fenêtre courte, il tombe au sol."

4. Cubes bonus, score et vitesse

Les cubes bonus donnent maintenant :

- 1 point ;
- un message en haut de l'écran ;
- une augmentation de vitesse pour le joueur qui ramasse le cube.

Extrait :

```
if (cScore != null) {
    isCollected = true;
    enabled = false;
    cScore.AddScore(Points);
}
```

```

    CharacterController controller = FindCollectorController(other, cScore);
    if (controller != null) {
        controller.ApplyBonusSpeedBoost();
    }
    ScorePanelHUD.ShowPickupMessage(controller);
}

```

Le booléen `isCollected` évite qu'un cube soit compté plusieurs fois si plusieurs collisions arrivent au même moment.

La vitesse augmente progressivement :

```

public float BonusSpeedIncrement = 0.5f;
public float MaxBonusSpeedMultiplier = 2f;
public float BonusSpeedDuration = 5f;

```

Quand un joueur ramasse un cube, le multiplicateur augmente, mais reste limité. Cela évite qu'un joueur devienne trop rapide et casse l'équilibre.

5. Génération de cubes supplémentaires

Des cubes bonus supplémentaires sont générés automatiquement au lancement de la scène.

Le script `ExtraBonusCubes` place les cubes dans une zone définie :

```

public int BonusCount = 8;
public int RandomSeed = 12345;
public Vector2 AreaCenter = new Vector2(250f, 255f);
public Vector2 AreaSize = new Vector2(42f, 42f);
public float MinimumDistanceBetweenCubes = 4f;

```

La génération vérifie que les cubes ne sont pas trop proches :

```

bool CanPlaceAt(Vector3 position, Vector3[] existingPositions, int placedCount)
{
    float minDistanceSqr = MinimumDistanceBetweenCubes * MinimumDistanceBetweenCubes;
    for (int i = 0; i < placedCount; i++) {
        Vector3 diff = position - existingPositions[i];
        diff.y = 0f;
        if (diff.sqrMagnitude < minDistanceSqr) {
            return false;
        }
    }

    return true;
}

```

Point à dire à l'oral : on utilise la distance au carré (`sqrMagnitude`) pour éviter un calcul de racine carrée inutile. C'est une petite optimisation classique.

6. Interface de score

Un panneau de score est créé automatiquement au lancement de la scène. Il affiche le nombre de joueurs et leur score.

Exemple :

```

Joueurs : 2 / 6
Joueur 1 : 1
Joueur 2 : 0

```

Le panneau n'est plus codé uniquement pour deux joueurs. Il dépend du nombre de skins disponibles :

```
public int MaxDisplayedPlayers = CharacterSkinCatalog.SkinCount;
```

Cela permet de garder une cohérence : si 6 skins existent, alors la limite affichée est 6.

Un message apparaît aussi quand un joueur ramasse un cube :

```
pickupMessageText.text = GetPlayerDisplayName(player.Player) + " a ramassé un cube";
pickupMessageText.enabled = true;
pickupMessageUntil = Time.time + PickupMessageDuration;
```

7. Voitures pilotables

Les voitures peuvent être utilisées par les joueurs. Le joueur peut entrer dans une voiture proche, la conduire, puis en sortir.

Quand un joueur entre dans une voiture :

- la voiture s'arrête ;
- le joueur devient enfant du siège de la voiture ;
- le personnage est caché ;
- le Rigidbody du joueur devient kinematic.

Extrait :

```
public void EnterCar(DrivableCar car)
{
    currentCar = car;
    isDriving = true;
    rb.isKinematic = true;
    SetCharacterVisible(false);
    transform.SetParent(car.Seat, false);
    transform.localPosition = Vector3.zero;
    transform.localRotation = Quaternion.identity;
}
```

Quand le joueur sort, la voiture peut rester garée :

```
public bool ParkAfterDriverExit = true;
```

Cela évite que la voiture reparte toute seule après la sortie du joueur.

8. Circulation autonome, obstacles et feux rouges

Les voitures qui roulent automatiquement doivent respecter certaines règles :

- s'arrêter au feu rouge ;
- s'arrêter devant un obstacle ;
- ne pas considérer les cubes bonus comme des obstacles ;
- klaxonner si elles sont bloquées derrière une voiture ou un personnage.

Extrait :

```
void DriveAutonomously()
{
    bool blockedByObstacle = ShouldStopForObstacleAhead(Mathf.Max(CarLookAhead,
ObstacleLookAhead));
```

```

if (ShouldStopForRedLight() || blockedByObstacle) {
    stoppedByTraffic = true;
    SetSceneAnimationPlaying(false);
    SetEngineSoundPlaying(false);
    StopNow();

    if (blockedByObstacle) {
        Honk();
    }
    return;
}

stoppedByTraffic = false;
SetSceneAnimationPlaying(true);
SetEngineSoundPlaying(true);
}

```

Pour ignorer les cubes :

```

if (hit.collider.GetComponentInParent<Bonus>() != null) { continue; }

```

Point à dire à l'oral : on distingue les voitures autonomes des voitures conduites par le joueur. Une voiture conduite par un joueur garde sa liberté, comme dans GTA. Les règles de circulation s'appliquent surtout aux voitures automatiques.

9. Sons de ville et sons de voitures

Plusieurs sons ont été ajoutés :

- ambiance de ville ;
- voix de foule ;
- moteur de voiture ;
- klaxon généré par code.

Les sons MP3 sont chargés depuis :

```

Assets/Demos/MetaVerse/Sound

```

Le chargement est centralisé dans `MetaVerseSoundLibrary` :

```

string path = Path.Combine(Application.dataPath, "Demos", "MetaVerse", "Sound", fileName);
string url = "file://" + path.Replace("\\", "/");

using (UnityWebRequest request = UnityWebRequestMultimedia.GetAudioClip(url, AudioType.MPEG))
{
    yield return request.SendWebRequest();
    AudioClip clip = DownloadHandlerAudioClip.GetContent(request);
    clips[fileName] = clip;
}

```

Point à dire à l'oral : on évite de charger le même son plusieurs fois grâce au dictionnaire de cache.

Pour les voitures, le son moteur suit la voiture animée :

```

Transform GetAudioAnchor()
{
    if (sceneAnimation != null) {
        return sceneAnimation.transform;
    }
}

```

```

    return transform;
}

```

Cela corrige le problème où le son restait au point de départ pendant que la voiture animée bougeait.

Le son moteur peut aussi commencer plus loin dans le MP3 :

```

public float EngineSoundStartOffset = 2f;

float GetEngineStartTime()
{
    return Mathf.Clamp(EngineSoundStartOffset, 0f, Mathf.Max(0f, engineSource.clip.length - 0.01f));
}

```

Cela permet de couper le début du fichier audio si les premières secondes ne sont pas naturelles.

10. Skins aléatoires sans doublon

Les joueurs reçoivent un skin aléatoire au lancement de la map. Les skins disponibles sont centralisés :

```

public const int SkinCount = 6;

static readonly CharacterSkinInfo[] Skins = {
    new CharacterSkinInfo("barbarian", "Barbare",
        "Assets/Demos/MetaVerse/Prefabs/barbarian.prefab"),
    new CharacterSkinInfo("druid", "Druide", "Assets/Demos/MetaVerse/Prefabs/druid.prefab"),
    new CharacterSkinInfo("engineer", "Ingenieur",
        "Assets/Demos/MetaVerse/Prefabs/engineer.prefab"),
    new CharacterSkinInfo("knight", "Chevalier",
        "Assets/Demos/MetaVerse/Prefabs/knight.prefab"),
    new CharacterSkinInfo("mage", "Mage", "Assets/Demos/MetaVerse/Prefabs/mage.prefab"),
    new CharacterSkinInfo("rogue", "Voleur", "Assets/Demos/MetaVerse/Prefabs/rogue.prefab"),
};

```

Les skins sont mélangés :

```

for (int i = indexes.Count - 1; i > 0; i--) {
    int swapIndex = Random.Range(0, i + 1);
    int current = indexes[i];
    indexes[i] = indexes[swapIndex];
    indexes[swapIndex] = current;
}

```

Puis un skin différent est appliqué à chaque joueur.

Un problème rencontré : les prefabs KayKit bruts mettaient les personnages en T-pose car ils n'avaient pas d'Animator Controller. La correction a été d'utiliser les prefabs MetaVerse déjà configurés avec les animations.

11. États réseau

Des classes d'état ont été ajoutées pour préparer la synchronisation réseau.

État joueur :

```

public class PlayerState

```

```
{
    public string Id;
    public string Name;
    public float X;
    public float Y;
    public float Z;
    public float RotY;
    public int Score;
    public string SkinId;
}
```

État bonus :

```
public class BonusState
{
    public string Id;
    public float X;
    public float Y;
    public float Z;
    public bool IsCollected;
}
```

Le monde contient les joueurs et les bonus :

```
public Dictionary<string, PlayerState> Players = new Dictionary<string, PlayerState>();
public Dictionary<string, BonusState> Bonuses = new Dictionary<string, BonusState>();
```

La limite de joueurs dépend des skins :

```
public int MaxPlayers = CharacterSkinCatalog.SkinCount;
```

Intérêt : on évite d'avoir plus de joueurs que de designs disponibles, ce qui respecte la règle "jamais deux fois le même skin".

12. Problèmes rencontrés et corrections importantes

Voitures stationnées qui avançaient

Correction : distinguer les voitures qui ont une animation de scène et les voitures garées. Les voitures garées ne doivent pas rouler automatiquement.

Voitures qui traversaient mal la logique d'obstacles

Correction : les voitures autonomes s'arrêtent devant les personnages et les autres voitures, mais ignorent les cubes bonus.

Son de voiture qui ne suivait pas

Correction : placer l'AudioSource sur le `Transform` animé, pas seulement sur le parent de la voiture.

Bruit moteur seulement au début

Correction : garder en mémoire que le moteur doit jouer, même si le MP3 n'est pas encore chargé :

```
bool wantsEngineSound;
```

Puis relancer le son quand le clip est prêt :

```
engineSource.clip = loadedClip;  
SetEngineSoundPlaying(wantsEngineSound);
```

T-pose sur les skins

Correction : utiliser les prefabs MetaVerse animés au lieu des prefabs KayKit bruts.

Score qui augmentait trop

Correction : ajouter ``isCollected`` et désactiver le script du bonus dès la collecte pour éviter plusieurs validations.

13. Ce que je peux dire à l'oral

Phrase d'introduction possible :

"Nous avons transformé une scène de démonstration Unity en une scène plus jouable, avec des personnages contrôlables, des bonus, un score, des véhicules, de la circulation autonome, des sons et une base d'état réseau."

Phrase sur la caméra :

"Le déplacement dépend de la caméra, ce qui rend le contrôle plus naturel. Le joueur avance dans la direction qu'il regarde."

Phrase sur les voitures :

"Les voitures autonomes respectent les obstacles et les feux, mais les voitures conduites par les joueurs gardent une liberté totale. Cela sépare clairement l'IA de circulation du gameplay joueur."

Phrase sur les sons :

"Les sons sont chargés dynamiquement depuis le dossier Sound. Les AudioSource des voitures sont attachées au bon objet pour que le son suive la voiture en mouvement."

Phrase sur le réseau :

"Les états PlayerState, BonusState et WorldState permettent de préparer la synchronisation : position, rotation, score, skin choisi et bonus collectés."

14. Améliorations possibles

Pour aller plus loin, on pourrait :

- synchroniser complètement les skins côté serveur ;
- ajouter une vraie interface de sélection ou d'attente multijoueur ;
- remplacer les sons MP3 par des boucles audio mieux découpées ;
- ajouter des animations de montée et descente de voiture ;
- ajouter un vrai système de santé ;
- sauvegarder les scores ;
- améliorer l'IA des voitures avec un système de chemins plus propre.

15. Conclusion

Le projet montre plusieurs compétences Unity :

- scripts C# organisés en composants ;
- utilisation de Rigidbody et collisions ;
- gestion de l'UI en runtime ;
- chargement de sons ;
- gestion d'états réseau ;
- génération procédurale simple ;
- résolution de bugs concrets liés à l'animation, au son et à la physique.

La partie la plus intéressante à présenter est la cohérence entre les systèmes : les cubes influencent le score et la vitesse, les voitures interagissent avec les joueurs et les obstacles, les sons suivent les objets, et la limite de joueurs dépend directement du nombre de skins disponibles.